

Kursabend 4

Dateiverarbeitung und Praxis-Workshop

Daniel Senften

Version 1.0, 2025-10-15

Table of Contents

1. Einführung	1
2. Dateiverarbeitung: Die Grundlagen	2
2.1. Dateien lesen	2
2.2. Dateien schreiben	2
2.3. Dateien zeilenweise lesen	2
3. Übungen	3
3.1. Level 1: Aufwärmen	3
3.2. Level 2: Herausforderung	4
3.3. Level 3: Für Schnelle	7
4. Tipps für erfolgreiches Üben	9
5. Zusätzliche Ressourcen	10
6. Reflexionsfragen	11

Chapter 1. Einführung

Heute vertiefen wir die bisherigen Konzepte und lernen, wie wir mit Dateien arbeiten können. Der Fokus liegt auf praktischem Üben und der Anwendung aller bisher gelernten Konzepte.

Chapter 2. Dateiverarbeitung: Die Grundlagen

2.1. Dateien lesen

Um eine Datei zu lesen, verwenden wir die Funktion `open()` zusammen mit einem `with`-Statement:

```
# Datei lesen
with open('meine_datei.txt', 'r', encoding='utf-8') as datei:
    inhalt = datei.read()
    print(inhalt)
```

Das `with`-Statement stellt sicher, dass die Datei automatisch geschlossen wird.

2.2. Dateien schreiben

Um in eine Datei zu schreiben:

```
# Datei schreiben
with open('ausgabe.txt', 'w', encoding='utf-8') as datei:
    datei.write('Hallo Welt!\n')
    datei.write('Dies ist die zweite Zeile.\n')
```



Der Modus `'w'` überschreibt die Datei komplett! Verwende `'a'` zum Anhängen.

2.3. Dateien zeilenweise lesen

Oft möchten wir eine Datei Zeile für Zeile verarbeiten:

```
with open('meine_datei.txt', 'r', encoding='utf-8') as datei:
    for zeile in datei:
        zeile = zeile.strip() # Entfernt Leerzeichen und Zeilenumbrüche
        print(zeile)
```

Chapter 3. Übungen

3.1. Level 1: Aufwärmen

Diese Aufgaben wiederholen die Grundlagen und führen sanft in die Dateiverarbeitung ein.

Aufgabe 1.1: Zahlenraten mit Highscore

Erweitere das Zahlenraten-Spiel aus Kursabend 3 um eine Highscore-Funktion.

Anforderungen:

- Das Programm speichert die Anzahl der benötigten Versuche in einer Datei `highscore.txt`
- Beim Start wird der bisherige Rekord (wenigste Versuche) angezeigt
- Nach jedem Spiel wird geprüft, ob ein neuer Rekord aufgestellt wurde
- Der Rekord wird mit Datum gespeichert

Hilfestellung:

```
import random
from datetime import datetime

def lade_highscore():
    """Lädt den Highscore aus der Datei."""
    try:
        with open('highscore.txt', 'r', encoding='utf-8') as datei:
            # Dein Code hier
            pass
    except FileNotFoundError:
        return None # Noch kein Highscore vorhanden

def speichere_highscore(versuche):
    """Speichert einen neuen Highscore."""
    # Dein Code hier
    pass
```

Aufgabe 1.2: Einkaufsliste

Erstelle ein Programm zur Verwaltung einer Einkaufsliste.

Anforderungen:

- Menü mit folgenden Optionen:
 - Artikel hinzufügen
 - Liste anzeigen
 - Artikel löschen

- Liste speichern
- Liste laden
- Programm beenden
- Die Liste wird in `einkaufsliste.txt` gespeichert
- Beim Start wird automatisch versucht, eine existierende Liste zu laden

Beispielausgabe:

```
=== Einkaufsliste ===
1. Artikel hinzufügen
2. Liste anzeigen
3. Artikel löschen
4. Liste speichern
5. Liste laden
6. Beenden

Wähle eine Option: 1
Artikel: Milch
Milch wurde hinzugefügt.
```

Aufgabe 1.3: Notizen-App

Erstelle eine einfache Notizen-Anwendung.

Anforderungen:

- Benutzer kann Notizen erstellen
- Jede Notiz wird mit Zeitstempel in einer eigenen Datei gespeichert
- Dateiname: `notiz_YYYY-MM-DD_HH-MM-SS.txt`
- Alle Notizen können aufgelistet werden
- Eine bestimmte Notiz kann gelesen werden

3.2. Level 2: Herausforderung

Diese Aufgaben kombinieren mehrere Konzepte und erfordern mehr Planung.

Aufgabe 2.1: Textanalyse-Tool

Erstelle ein Programm, das eine Textdatei analysiert.

Anforderungen:

- Lies eine Textdatei ein (z.B. `beispieltext.txt`)
- Berechne und zeige an:

- Anzahl Zeilen
- Anzahl Wörter
- Anzahl Zeichen
- Durchschnittliche Wortlänge
- Das längste Wort
- Das häufigste Wort
- Erstelle einen Analysebericht in `analyse.txt`

Beispielausgabe:

```
=== Textanalyse ===
Datei: beispieltext.txt

Zeilen: 42
Wörter: 387
Zeichen: 2.145
Durchschnittliche Wortlänge: 5.5 Zeichen
Längstes Wort: Programmierung (14 Zeichen)
Häufigstes Wort: Python (23 mal)
```

Hilfestellung:

```
def analysiere_text(dateiname):
    """Analysiert eine Textdatei."""
    with open(dateiname, 'r', encoding='utf-8') as datei:
        zeilen = datei.readlines()

    # Zähle Zeilen
    anzahl_zeilen = len(zeilen)

    # Zähle Wörter
    alle_woerter = []
    for zeile in zeilen:
        woerter = zeile.split()
        alle_woerter.extend(woerter)

    # Weitere Berechnungen...
    pass
```

Aufgabe 2.2: CSV-Verarbeitung

Erstelle ein Programm, das einfache CSV-Daten (Comma-Separated Values) verarbeitet.

Anforderungen:

- Lies eine CSV-Datei mit Produktdaten ein (`produkte.csv`)

- Format: **Name,Preis,Anzahl**
- Berechne:
 - Gesamtwert aller Produkte (Preis × Anzahl)
 - Durchschnittspreis
 - Teuerstes Produkt
 - Günstigstes Produkt
- Erstelle einen Bericht

Beispiel produkte.csv:

```
Apfel,1.50,10
Banane,0.80,15
Orange,2.00,8
Birne,1.80,12
```

Hilfestellung:

```
def lese_csv(dateiname):
    """Liest CSV-Datei und gibt Liste von Produkten zurück."""
    produkte = []
    with open(dateiname, 'r', encoding='utf-8') as datei:
        for zeile in datei:
            zeile = zeile.strip()
            teile = zeile.split(',')

            produkt = {
                'name': teile[0],
                'preis': float(teile[1]),
                'anzahl': int(teile[2])
            }
            produkte.append(produkt)

    return produkte
```

Aufgabe 2.3: Log-File-Analyzer

Erstelle ein Programm, das Log-Dateien nach bestimmten Mustern durchsucht.

Anforderungen:

- Lies eine Log-Datei ein (**server.log**)
- Suche nach Zeilen, die bestimmte Schlüsselwörter enthalten (z.B. "ERROR", "WARNING")
- Zähle die Vorkommen
- Erstelle einen Bericht mit den gefundenen Zeilen

- Speichere nur die Fehlerzeilen in `errors.log`

Beispiel `server.log`:

```
2025-10-15 10:23:45 INFO Server gestartet
2025-10-15 10:24:12 INFO Verbindung hergestellt
2025-10-15 10:25:33 WARNING Langsame Antwortzeit
2025-10-15 10:26:01 ERROR Verbindung fehlgeschlagen
2025-10-15 10:27:15 INFO Neustart erfolgreich
```

3.3. Level 3: Für Schnelle

Diese Aufgaben sind anspruchsvoller und kombinieren alle bisherigen Konzepte.

Aufgabe 3.1: Vokabeltrainer

Erstelle einen Vokabeltrainer mit Statistik-Funktion.

Anforderungen:

- Vokabeln werden aus `vokabeln.txt` geladen (Format: `Deutsch:Englisch`)
- Das Programm fragt zufällige Vokabeln ab
- Statistik wird geführt:
 - Anzahl richtige Antworten
 - Anzahl falsche Antworten
 - Erfolgsquote
- Schwierige Vokabeln (oft falsch) werden häufiger abgefragt
- Statistik wird in `statistik.txt` gespeichert

Beispiel `vokabeln.txt`:

```
Haus:house
Auto:car
Baum:tree
Buch:book
Tisch:table
```

Aufgabe 3.2: Kontaktverwaltung

Erstelle eine einfache Kontaktverwaltung.

Anforderungen:

- Kontakte speichern mit: Name, Telefon, E-Mail

- Funktionen:
 - Kontakt hinzufügen
 - Kontakt suchen (nach Name)
 - Kontakt bearbeiten
 - Kontakt löschen
 - Alle Kontakte anzeigen
- Speicherung in **kontakte.txt**
- Format: Eine Zeile pro Kontakt, Felder getrennt durch |

Beispiel kontakte.txt:

```
Max Mustermann|079 123 45 67|max@example.com  
Anna Schmidt|078 987 65 43|anna@example.com
```

Chapter 4. Tipps für erfolgreiches Üben



- **Plane zuerst:** Überlege dir die Schritte, bevor du mit dem Programmieren beginnst
- **Klein anfangen:** Beginne mit einer einfachen Version und erweitere sie dann
- **Testen:** Teste dein Programm mit verschiedenen Eingaben
- **Fehler sind normal:** Lass dich nicht entmutigen, Fehler gehören zum Lernen dazu
- **Hilfe holen:** Frage deine Mitstudierenden oder den Dozenten

Chapter 5. Zusätzliche Ressourcen

- Python-Dokumentation zu Dateiverarbeitung: <https://docs.python.org/de/3/tutorial/inputoutput.html#reading-and-writing-files>
- Tutorial zu `with`-Statement: <https://realpython.com/python-with-statement/>

Chapter 6. Reflexionsfragen

Nach dem Üben:

1. Welche Aufgabe war am schwierigsten? Warum?
2. Was hast du Neues gelernt?
3. Welches Konzept möchtest du noch vertiefen?
4. Wie könntest du die gelernten Konzepte im Alltag anwenden?